

Laborprojekt – Versuch 8

Einführung

In diesem Versuch erweitern Sie die in den vorherigen Laboren erstellten Komponenten um einen generischen Shifter und eine ALU. Sie greifen auf die aktualisierte *Constant_Package.vhdl* aus Moodle zu und testen die bereitgestellten VHDL-Gerüste mit den passenden Testbenches.

Lernziele

Die Studierenden

- können die Funktionsweise von logischen und arithmetischen Shiftern erklären und einen generischen Shifter (links/rechts) in VHDL eigenständig umsetzen.
- entscheiden zwischen Prozess- und Datenflussbeschreibung und können diese Varianten anforderungsspezifisch umsetzen.
- implementieren eine generische ALU mit Konstanten/Generics, welche XOR, OR, AND, ADD, SUB, SRL, SRA und SLL unterstützt.
- testen ihre VHDL-Designs mit GHDL und den bereitgestellten Testbenches für unterschiedliche Datenbreiten nachvollziehbar.

Vorbereitung

- Ordner `versuch8` ins Home-Verzeichnis kopieren und die aktuelle *Constant_Package.vhdl* aus Moodle verwenden.
- Testbenches `my_shifter_tb*.vhdl` und `my_alu_tb.vhdl` bereitlegen; GHDL mit `-std=08` nutzen.

Aufgaben

Aufgabe 1: Generischer Shifter

Ein generischer Shifter ist eine Schaltung, die dazu dient, Bits in einem Datenwort um eine bestimmte Anzahl von Positionen zu verschieben. Der Shifter kann nach links (*Shift Left*) oder nach rechts (*Shift Right*) operieren. Zusätzlich kann der Shifter die Bits, die durch das Verschieben freigelegt werden, mit Nullen oder dem Wert des am weitesten verschobenen Bits (bei arithmetischem Shift) auffüllen.

- **SLL (Shift Left Logical):** Logischer Linksshift, füllt die freigelegten Positionen mit Nullen.
 - **SRL (Shift Right Logical):** Logischer Rechtsshift, füllt die freigelegten Positionen mit Nullen.
 - **SRA (Shift Right Arithmetic):** Arithmetischer Rechtsshift, füllt die freigelegten Positionen mit dem Wert des ursprünglichen ganz links liegenden Bits (Sign-Bit).
- a) Ergänzen Sie im Folgenden die Entity des generischen Shifters in (*my_shifter.vhdl*) um die Ports, wie in Abbildung 1 dargestellt. Der Parameter *p_shift_type* gibt dazu die Art der Shiftoperation an (0 := logisch und 1 := arithmetisch), während der Parameter *p_shift_dir* die Richtung (0 := links und 1 := rechts) angibt.

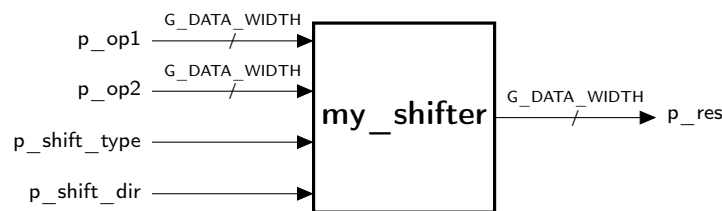


Abbildung 1: Entity *my_shifter*

- b) Ergänzen Sie die Verhaltensarchitektur der Entity *my_shifter* um einen Prozess, der auf alle notwendigen PORTs bzw. Signale sensitiv ist und die jeweilige Shiftoperation in Abhängigkeit von *p_shift_type*, *p_shift_dir* und dem Signal *s_shamtInt* ausführt. Hierbei steht "shamt" für "shift amount". Das Signal *s_shamtInt* wird aus dem Parameter *p_op2* generiert und ist definiert über den Bereich von 0 bis maximal *G_DATA_WIDTH*. Dazu wird der Parameter *p_op2* als unsigned interpretiert und nur die letzten $\log_2(G_DATA_WIDTH)$ Bits betrachtet.
- c) Erstellen Sie ein Bash-Skript und testen Sie Ihre Entities erfolgreich mit der vorgegebenen Testbench *my_shifter_tb.vhdl* mit einer Länge von 4 Bit und 8 Bit mit Hilfe der Konstanten *C_DWG*.

- d) Laden Sie alle erstellten und angepassten Dateien als Zip-File *Name_Vorname_V8_A1.zip* in Moodle hoch.

Abnahme *Standard VHDL, Konstanten, Packages, Generics, Shifter*

Aufgabe 2: Arithmetic Logical Unit (ALU).....

Im Rahmen des Laborprojekts sollen Sie eine generische ALU durch Verwendung dedizierter generischer Entitäten für jede Operation, um eine möglichst hohe Wiederverwendung zu gewährleisten, erstellen. Dabei werden alle verfügbaren Operationen parallel berechnet, jedoch nur das Ergebnis der ausgewählten Operation an die Ausgabe der ALU (*p_alu_out*) weitergeleitet.

Die ALU soll unabhängig von einer Clock arbeiten. Ändern sich die Eingänge, so wird also die Operation sofort durchgeführt und das Ergebnis an die Ausgabe der ALU (*p_alu_out*) weitergeleitet. Hierfür ist es hilfreich mit einem "with...select"-Statement zu arbeiten. Ein unvollständiges Beispiel könnte so aussehen:

```
1      with P_ALU_OP select
2          P_ALU_OUT <= s_res3 when C_AND_OP ,
3          ...
4          (others => '0') when others;
5
```

Da generische Selektoren (hier: *P_ALU_OP*) erst ab VHDL-2008 unterstützt werden, muss in GHDL diese Version gewählt werden: Dies erfolgt mit dem Parameter *-std=08* in jedem GHDL-Kommando.

- Beispiel: `ghdl -a -std=08 my_gen_and.vhdl`

Als Codierungen für die einzelnen Operationen sollen wieder Konstanten aus dem *Constant_Package* verwendet werden. **Achten Sie hierbei darauf, dass für diesen Versuch die aktualisierte Datei *Constant_Package.vhdl* auf Moodle bereitsteht.** Die Codierungen der ALU-Operationen (*p_alu_op*) wurden hierbei so gewählt, dass das äußerst linke Bit, also das most significant Bit (*p_alu_op(G_OP_WIDTH-1)*) für das Signal des Shift-Type (*s_shift_type*) und des Carry-in (*s_cln*) genutzt werden soll.

- a) Ergänzen Sie nun die Ports der Entity *my_alu* wie schematisch in Abb. 2 dargestellt.
- b) Kopieren Sie die generischen logischen Funktionen (AND, OR, XOR) aus Versuch 5, sowie Ihren generischen Addierer/Subtrahierer (ADD, SUB) aus Versuch 7 und alle dafür benötigten Entitäten in den Ordner für Versuch 8 und erweitern Sie die Datei *my_alu.vhdl* zu einer Arithmetic Logical Unit (ALU), so dass die ALU die Operationen

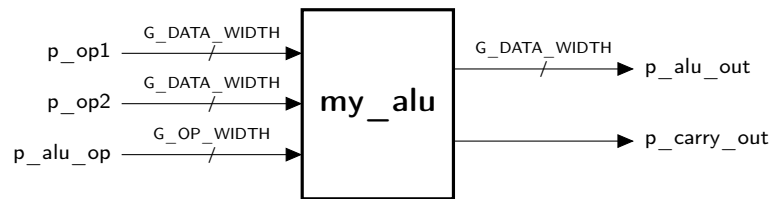


Abbildung 2: Entity my_alu

XOR, OR, AND, ADD, SUB und die Shift-Befehle SRA, SRL und SLL (Aufgabe 1) unterstützt. Passen Sie die Signale an, so dass diese mit Abbildung 2 übereinstimmen. Achten Sie auf die Konstanten in der **vorgegebenen** Datei *Constant_Package.vhdl*.

- c) Ergänzen Sie nun alle notwendigen Signale für alle Operationen, implementieren Sie die Logik der ALU, erstellen Sie ein Bash-Skript und testen Sie Ihre ALU mit der Testbench *my_alu_tb.vhdl* erfolgreich.
- d) Laden Sie alle erstellten und angepassten Dateien als Zip-File *Name_Vorname_V8_A2.zip* in Moodle hoch.

Abnahme *Standard VHDL, Konstanten, Packages, Generics, ALU*

Knobelaufgabe 1: Shifter als Datenfluss

Der Shifter in der Aufgabe 1 wurde als Prozess umgesetzt. Setzen Sie den generischen Shifter nun als Datenfluss um.

- Ergänzen Sie dazu die Datei *my_shifter.vhdl* um eine weitere Architektur mit Namen *dataflow*.
- Setzen Sie den generischen Shifter als Datenfluss (ohne Prozess und weitere Komponenten) um.
- Erstellen Sie ein Bash-Skript und überprüfen Sie die Umsetzung mit der Testbench *my_shifter_tb2* erfolgreich.
- Laden Sie alle erstellten und angepassten Dateien als Zip-File *Name_Vorname_V8_K.zip* in Moodle hoch.

Abnahme *Standard VHDL, Shifter als Datenfluss*

Abgabeübersicht

Aufgabe	Datei	Inhalt
A1	Name_Vorname_V8_A1.zip	• my_shifter.vhdl • aktuelle Constant_Package.vhdl • Bash-Skript (Tests mit 4/8 Bit)
A2	Name_Vorname_V8_A2.zip	• my_alu.vhdl • verwendete Basis-Komponenten • aktuelle Constant_Package.vhdl • Bash-Skript
K1	Name_Vorname_V8_K.zip	• my_shifter.vhdl (dataflow) • my_shifter_tb2.vhdl • Bash-Skript (Tests für 4/8 Bit)